Una tortuga te sigue

Descripción e introducción

Vas a construir un pequeño juego con ROS 2 y Python. En una ventana habrá dos tortugas: tú controlarás turtle1 con el teclado y turtle2 intentará seguirte. El proyecto se inspira en el juego de gato y ratón de este repositorio.

En la primera parte solo habrá movimiento y persecución. El simulador crea la primera tortuga y tú pedirás que aparezca la segunda. No habrá captura, puntuación ni mensajes de fin.

Qué aprenderás

Crear un paquete, recibir posiciones mediante suscriptores y enviar velocidades mediante un publicador. También aprenderás a arrancar varios programas y comprobar qué hace cada uno.

Qué necesitas saber

Variables, funciones, condicionales y un poco de Python. No necesitas haber visto programación orientada a objetos. Organiza tu programa con funciones y variables; no tienes que definir clases ni usar herencia.

Guion de trabajo

1. Comprobar el equipo (p. 2).
2. Abrir el simulador, crear turtle2 y probar el teclado (p. 3).
3. Crear el paquete y el archivo seguidor.py (p. 4).
4. Programar sus conexiones y funciones (p. 5).
5. Calcular el movimiento y ejecutar la prueba (p. 6).
6. Mejorar el arranque con un launch (p. 7).
7. Añadir otros retos cuando lo anterior funcione (p. 8).

Cómo usar el material

Este PDF contiene instrucciones y pistas, no la solución Python. Los bloques son comandos de terminal. Intenta resolver cada paso; si te atascas, consulta el archivo solucion.md, que contiene el programa y el launch completos.

1. Comprueba el equipo

Los comandos están pensados para Ubuntu 24.04, ROS 2 Jazzy y terminal Bash. Ejecuta las líneas una a una.

Comprueba que ROS está disponible

```
source /opt/ros/jazzy/setup.bash
ros2 --help
ros2 pkg executables turtlesim
```

Debes ver la ayuda y una lista que incluya turtlesim_node y turtle_teleop_key. Si no existe setup.bash, necesitas preparar ROS o usar la ruta de la distribución instalada. Pide ayuda antes de seguir.

Si faltan turtlesim o colcon

```
sudo apt update
sudo apt install ros-jazzy-turtlesim \
  python3-colcon-common-extensions
```

Estos comandos suponen que ROS Jazzy y su repositorio ya están configurados. Pueden pedir la contraseña del equipo. La barra \ permite continuar el comando en la siguiente línea.

Tres instrucciones útiles de terminal

source: prepara la terminal para encontrar ROS o tu paquete.
cd: cambia de carpeta.
mkdir -p: crea carpetas. El símbolo ~ representa tu carpeta personal.

Antes de abrir cada terminal nueva

Ejecuta source /opt/ros/jazzy/setup.bash. Esto afecta solo a esa terminal. Más adelante cargarás también el archivo install/setup.bash de tu workspace.

Comprobación: no avances hasta que puedas consultar los ejecutables de turtlesim. Necesitas además una sesión gráfica para que se abra su ventana.

2. Pon las dos tortugas

Terminal A: abre el simulador

```
source /opt/ros/jazzy/setup.bash
ros2 run turtlesim turtlesim_node
```

Deja el proceso abierto. Verás turtle1 en el centro. El simulador ya crea esa primera tortuga.

Terminal B: crea la segunda

```
source /opt/ros/jazzy/setup.bash
ros2 service call /spawn turtlesim/srv/Spawn \
  "{x: 1.0, y: 1.0, theta: 0.0, name: 'turtle2'}"
```

Aparecerá turtle2 cerca de una esquina, lejos de la posición inicial del jugador. Ejecuta este comando una sola vez por partida.

En la terminal B: abre el teclado

```
ros2 run turtlesim turtle_teleop_key
```

Haz clic en la terminal B y usa las flechas. Arriba/abajo hacen avanzar/retroceder; izquierda/derecha hacen girar. Mantén el foco en esa terminal, aunque mires la ventana del simulador.

Las palabras de ROS que necesitas

Nodo: un programa ROS en ejecución, como el simulador.
Topic: un canal por donde circulan mensajes.
Publicador: envía mensajes a un topic.
Suscriptor: recibe los mensajes de un topic.
Servicio: recibe una petición y devuelve una respuesta; /spawn crea una tortuga.

Comprobación: ves dos tortugas, mueves turtle1 y turtle2 está quieta. El programa que vas a escribir dará órdenes a turtle2.

3. Crea el paquete

Abre una terminal C. Trabaja en un workspace separado, una carpeta donde guardar y construir tu proyecto. Si ya creaste este paquete siguiendo la versión anterior, reutilízalo: no lo vuelvas a crear.

```
source /opt/ros/jazzy/setup.bash
mkdir -p ~/pilla_pilla_ws/src
cd ~/pilla_pilla_ws/src
ros2 pkg create pilla_pilla --build-type ament_python \
  --license Apache-2.0 \
  --dependencies rclpy turtlesim geometry_msgs
```

El comando prepara un paquete Python. rclpy aporta las funciones de ROS. turtlesim aporta las posiciones y el servicio de creación. geometry_msgs aporta los mensajes de velocidad.

Crea seguidor.py con tu editor

Guárdalo dentro de la carpeta Python del paquete, en esta ruta:

```
~/pilla_pilla_ws/src/pilla_pilla/pilla_pilla/seguidor.py
```

Hay dos carpetas llamadas pilla_pilla, una dentro de otra. Es correcto. El archivo setup.py está en la exterior; seguidor.py va en la interior, junto a __init__.py.

Empieza con funciones y variables

Prepara una función principal llamada main. Esta iniciará ROS con rclpy.init, creará un nodo llamado seguidor con rclpy.create_node, preparará sus conexiones y lo mantendrá atendiendo mensajes con rclpy.spin.

Añade dos variables para guardar las últimas posiciones: la del jugador y la del seguidor. Al principio no tendrás ninguna posición; usa None para representar «todavía no ha llegado».

Puedes compartir esas variables entre funciones. Si una función sustituye el valor de una variable definida fuera de ella, necesitarás declararla global en esa función. Es una opción sencilla para este primer ejercicio.

No necesitas crear una clase Node. Utilizarás un nodo que ROS ya sabe crear. En expresiones como mensaje.x, el punto significa «lee el campo x de este mensaje».

4. Qué debe tener seguidor.py

Dos suscriptores de posición

Suscriptor 1: se suscribe a /turtle1/pose, de tipo turtlesim.msg.Pose. Al recibir un mensaje, llama a una función que guarda la posición del jugador.

Suscriptor 2: se suscribe a /turtle2/pose, del mismo tipo. Llama a otra función que guarda la posición del seguidor. Estas funciones solo guardan datos; todavía no calculan movimiento.

Pista de API: usa nodo.create_subscription. Necesita el tipo de mensaje, el nombre del topic, la función que recibirá los mensajes y una profundidad de cola; utiliza 10. La función recibe el mensaje como argumento. ROS la llama por ti: eso es un callback.

Un publicador de velocidad

Publica en /turtle2/cmd_vel, usando geometry_msgs.msg.Twist. Sirve para mover solo a turtle2. Créalo con nodo.create_publisher, indicando tipo, topic y profundidad 10.

Cada mensaje de velocidad tendrá dos campos relevantes: linear.x para avanzar y angular.z para girar. Los demás pueden quedar a cero. Para enviarlo, usa la operación publish del publicador.

Un temporizador y una función de seguimiento

Prepara una función, por ejemplo seguir, que consulte las posiciones guardadas y calcule una orden. Un temporizador creado con nodo.create_timer debe llamarla cada 0,05 segundos. La función del temporizador no recibe un mensaje.

Primero comprueba que han llegado ambas posiciones. Si alguna sigue siendo None, sal de la función sin publicar nada. Así no intentas calcular con datos que aún no existen.

Los campos que vas a usar

En Pose, x e y son las coordenadas y theta es hacia dónde mira la tortuga. Los ángulos están en radianes: pi radianes equivalen a 180 grados.

No programes un suscriptor de teclado: turtle_teleop_key ya envía velocidades a /turtle1/cmd_vel. No necesitas servicios propios, árbitro ni mensajes de fin en esta parte.

5. Haz que siga y ejecútalo

Pistas para la función seguir

1. Calcula dx restando la x del seguidor a la x del jugador. Haz lo mismo con y para obtener dy.
2. Calcula la distancia con math.hypot(dx, dy) y la dirección hacia el jugador con math.atan2(dy, dx).
3. Resta la orientación actual del seguidor a esa dirección. Ese es el error de giro.
4. Para elegir el giro corto, aplica math.atan2 al seno y al coseno del error, en ese orden.
5. Crea un mensaje Twist. Para avanzar, usa el menor valor entre 1,0 y la distancia. Para girar, usa el error multiplicado por 4,0. Envía el mensaje con el publicador.

Esta regla reduce el avance cuando las tortugas están cerca. No es una captura: si el jugador se aleja, el seguidor vuelve a perseguirlo.

Registra el programa

En setup.py, busca la lista console_scripts dentro de entry_points. Añade una entrada que relacione el comando seguidor con la función main del módulo pilla_pilla.seguidor. La forma exacta de esa entrada está en solucion.md.

Construye desde la terminal C

```
cd ~/pilla_pilla_ws
colcon build --symlink-install --packages-select pilla_pilla
source install/setup.bash
ros2 run pilla_pilla seguidor
```

Prueba: vuelve al teclado de la terminal B. Mueve turtle1 y comprueba que turtle2 la sigue. Cuando esto funcione, has completado la primera parte. Para cerrar, pulsa Ctrl+C en A, B y C.

Si algo falla

Paquete no encontrado: carga ROS y el install/setup.bash de tu workspace.
Ejecutable no encontrado: revisa setup.py y reconstruye.
No sigue: comprueba que existen ambas tortugas y llegan ambas posiciones.
Teclado sin respuesta: activa su terminal.

6. Arranca con un launch

Un launch es un archivo que organiza el arranque de varios programas. Haz esta mejora en cuanto funcione la primera parte: te evitará repetir varios comandos.

Qué debe tener

Crea la carpeta launch dentro de la raíz del paquete, junto a setup.py. Dentro crea juego.launch.py. La función generate_launch_description debe devolver las acciones necesarias para iniciar:

1. El simulador: paquete turtlesim, ejecutable turtlesim_node.
2. La creación de turtle2: un proceso que ejecute el mismo comando de /spawn que ya probaste, con posición (1,1) y nombre turtle2.
3. Tu seguidor: paquete pilla_pilla, ejecutable seguidor.

Pista: usa launch_ros.actions.Node para los programas ROS y launch.actions.ExecuteProcess para el comando de creación. Esos elementos los proporciona ROS; no tienes que programar clases. Encontrarás una versión completa en solucion.md.

Instálalo y decláralo

Añade launch y launch_ros como dependencias de ejecución en package.xml. En setup.py, añade el archivo de launch a data_files, dentro de la carpeta instalada share/pilla_pilla/launch. Reconstruye y vuelve a cargar install/setup.bash.

Prueba con un mundo nuevo

Primero cierra el simulador, el teclado y el seguidor anteriores. En una terminal preparada con ROS y tu workspace, ejecuta:

```
ros2 launch pilla_pilla juego.launch.py
```

En otra terminal preparada con ROS, abre el teclado:

```
ros2 run turtlesim turtle_teleop_key
```

Mantén el teclado separado para que pueda leer las flechas. El proceso de /spawn espera al servicio; tu seguidor espera a recibir ambas posiciones. Así soportan que el simulador tarde un poco en arrancar.

No ejecutes además el comando manual de /spawn: el launch ya crea turtle2. Al pulsar Ctrl+C en el launch, se cierran los procesos que ha iniciado; cierra también la terminal del teclado.

7. Añade mejoras poco a poco

Captura y tiempo de supervivencia

Cuando la distancia sea menor que 0,6, publica un Twist a cero y guarda un indicador de fin. A partir de entonces, no envíes más órdenes. Muestra «¡Te han pillado!» una sola vez. Para contar segundos, guarda time.monotonic al empezar y calcula la diferencia al capturar.

Dificultad y parámetros

Prueba otras velocidades máximas y distancias de captura. Después declara parámetros con nodo.declare_parameter y lee sus valores con nodo.get_parameter. Más adelante puedes reunirlos en un YAML, como en el repositorio original.

Creación desde Python y posición más lejana

Sustituye el comando /spawn por un programa creador.py. Debe tener un cliente de /spawn, tipo turtlesim.srv.Spawn, esperar al servicio, enviar x, y, theta y name y esperar la respuesta. Para elegir la salida, escucha /turtle1/pose y escoge la más lejana de cuatro esquinas interiores. Registra su ejecutable y úsalo en el launch.

Árbitro independiente y parada del jugador

Crea juez.py con dos suscriptores Pose y un publicador Bool en /game_over. El seguidor escucha ese aviso y se para. Añade std_msgs a las dependencias. Para parar al jugador, remapea la salida del teclado a /jugador/intencion y añade un nodo que la reenvíe a /turtle1/cmd_vel solo mientras la partida siga activa. El teclado estándar no conoce el fin de la partida.

Cuida el repositorio

Ignora /build/, /install/, /log/ y __pycache__/ en .gitignore. Guarda en Git el código fuente y la configuración. Intenta una mejora cada vez y comprueba que lo anterior sigue funcionando.

Documentación y solución

La solución de la primera parte y del launch recomendado está en solucion.md. El resto son retos de ampliación. Para consultar las herramientas, usa estos tutoriales oficiales de ROS 2 Jazzy:

Turtlesim y teclado · docs.ros.org

Crear un paquete Python · docs.ros.org

Instalar Jazzy en Ubuntu · docs.ros.org